

src/components/launch/LaunchContext.tsx

Kind: added · Baseline: a9c34bfb03fe · Target: NovaDiff · Generated: 2026-05-20T05:14:39.126Z

Overview

`src/components/launch/LaunchContext.tsx` introduces a React context for the launch sequence. It defines the `LaunchContextValue` interface, creates a provider component, and exports a hook to consume the context.

Key changes

- **Imports** – added `createContext` / `useContext` from React (R1) and the `LaunchPhase` type from `../../app/launchSequence` (R2).
- **Interface** – `LaunchContextValue` exposes `phase: LaunchPhase`, `skipSequence: boolean`, and `brandReveal: boolean` (R4-R7).
- **Context** – instantiated with defaults: `phase: "ready"`, `skipSequence: true`, `brandReveal: true` (R10-R14).
- **Provider** – `LaunchProvider` accepts a `value` prop of type `LaunchContextValue` and renders `<LaunchContext.Provider>` (R16-R24).
- **Hook** – `useLaunch()` returns the current context value via `useContext(LaunchContext)` (R26-R28).

Impact

- Components that consume launch state must import `useLaunch` or be wrapped in `LaunchProvider`.
- The new context centralizes launch-related state, eliminating duplicated logic across components.
- Context updates propagate only to consumers; the default values are lightweight.
- No breaking API changes; existing code remains unaffected unless it starts importing the new context.

Risks & follow-ups

- **Missing provider** – verify that all components using `useLaunch()` are rendered within a `LaunchProvider`.
- **Phase enum alignment** – ensure that the string `"ready"` matches a valid `LaunchPhase` value; otherwise TypeScript will error.
- **Default values** – confirm that `skipSequence: true` and `brandReveal: true` match the intended initial UI state.
- **Import paths** – check that the relative import `../../app/launchSequence` resolves correctly in all environments.